“Normalformen in Datenbanken in Theorie und Praxis”

Prof. Dr. Asis Hallab

Musterlösungen für die Übungsaufgaben zur Vorlesung

1. Aufgabe: Generieren Sie Daten

Problem

Zwischen `cities` und `countries` existiert eine zyklische Fremdschlüsselabhängigkeit:

```
cities.country_id -> countries.id
countries.(id, capital_name) -> cities.(country_id, name)
```

Dadurch können Datensätze nicht einfach in beliebiger Reihenfolge eingefügt werden.

Lösungsmöglichkeiten

Möglichkeit 1: Deferred Foreign Keys

```
FOREIGN KEY (id, capital_name)
REFERENCES cities(country_id, name)
DEFERRABLE INITIALLY DEFERRED
```

Die Fremdschlüsselprüfung erfolgt erst beim `COMMIT`.

Möglichkeit 2: Foreign Keys temporär deaktivieren

Diese Lösung ist spezifisch für SQLite.

```
PRAGMA foreign_keys = OFF;
-- INSERT Statements
PRAGMA foreign_keys = ON;
```

Beispiel INSERT

```
INSERT INTO countries (
    id,
    name,
    area,
    languages,
    capital_name,
    capital_current_governor
)
VALUES (
    1,
    'Deutschland',
    357588,
    'Deutsch',
    'Berlin',
```

```
    'Kai Wegner'
);
```

2. Aufgabe: Probleme aus Verletzungen der ersten Normalform

Problem

Das Attribut:

`governors TEXT`

enthält mehrere Werte in einem Feld.

Beispiel:

`'Hanno Benz (2023-heute), Jochen Partsch (2017-2023)'`

Dies verletzt die erste Normalform.

Problem bei Abfragen

Eine echte Liste von Bürgermeistern liegt nicht vor.

SQL muss den String zunächst parsen.

Beispiel

```
SELECT governors
FROM cities
WHERE name = 'Darmstadt';
```

liefert nur einen langen Textstring.

Lösungsmöglichkeiten

Shell

```
tr ',' '\n'
```

awk

```
awk -F',' '{for(i=1;i<=NF;i++) print $i}'
```

Python

```
text.split(',')
```

Korrekte Modellierung

```
CREATE TABLE city_governors (
    city_name TEXT NOT NULL,
    country_id INTEGER NOT NULL,
    governor_name TEXT NOT NULL,
    from_date TEXT NOT NULL,
    PRIMARY KEY (city_name, country_id, from_date)
);
```

3. Aufgabe: Probleme aus Verletzungen der zweiten Normalform

Speicherabschätzung

Annahmen:

- 10 Unicode Zeichen
- 4 Bytes pro Zeichen

Speicher pro Ländername:

```
10 * 4 = 40 Bytes
```

Ländername kommt vor:

- `name` in `countries`
- `country_name` in `cities`
- `country_name` in `ports`

Eine Million Städte:

```
1.000.000 * 40 * 3 Bytes
= 120.000.000 Bytes
equals approximately 120 MB
```

Problem

Würde der Name eines Landes nur in einer Tabelle gespeicher, wäre der Speicherbedarf um ein Drittel kleiner. Inkonsistenzen könen leicht durch unvollständige updates entstehen.

Beispiel UPDATE

```sql
BEGIN;
    UPDATE cities SET country_name = 'Bundesrepublik Deutschland'
        WHERE country_name = 'Deutschland';
    UPDATE countries SET name = 'Bundesrepublik Deutschland'
        WHERE name = 'Deutschland';
    UPDATE ports SET country_name = 'Bundesrepublik Deutschland'
        WHERE country_name = 'Deutschland';
COMMIT;
```

Korrektur

```sql
CREATE TABLE countries (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);
```

`cities` enthält dann nur noch:

```sql
country_id INTEGER NOT NULL
```

4. Aufgabe: Probleme aus Verletzungen der dritten Normalform

Funktionale Abhängigkeiten

```
id -> capital_name
capital_name -> capital_current_governor
```

Damit transitiv:

```
id -> capital_current_governor
```

Beispiel UPDATE

```sql
BEGIN;
    UPDATE countries SET capital_current_governor = 'Max Beispiel'
        WHERE capital_name = 'Berlin';
    UPDATE cities SET governors = 'Max Beispiel'
        WHERE name = 'Berlin';
COMMIT;
```

Problem

Die Information über den Bürgermeister gehört semantisch zur Hauptstadt, nicht direkt zum Land.

Mögliche Probleme:

- Inkonsistenzen
- Mehrfachpflege
- Redundanz
- Änderungsanomalien

Korrektur

```sql
CREATE TABLE city_governors (
    city_name TEXT NOT NULL,
    country_id INTEGER NOT NULL,
    governor_name TEXT NOT NULL,
    from_date TEXT NOT NULL,
    PRIMARY KEY (city_name, country_id, from_date)
);
```

5. Aufgabe: Probleme aus Verletzungen der BCNF

Tabelle `city_postcodes`

Relation:

```
city_postcodes(city_name, postcode, district)
```

Funktionale Abhängigkeiten

```
(city_name, postcode) -> district
postcode -> district
```

Kandidatenschlüssel

```
(city_name, postcode)
```

Problem

`postcode` bestimmt `district`, ist aber kein Superschlüssel.

Damit liegt eine BCNF-Verletzung vor.

Tabelle `ports`

Relation:

`ports(name, river_id, river_mouth_coordinates, country_id)`

Funktionale Abhängigkeit

`river_id -> river_mouth_coordinates`

`river_id` ist kein Superschlüssel.

Problem

Die Flussmündung ist Eigenschaft des Flusses, nicht des Hafens.

6. Aufgabe: 1 NF

Verletzungen

Tabelle `cities`

`governors TEXT`

Korrektur *Siehe `city_governors` oben.*

Tabelle `countries`

`languages TEXT`

Korrektur

```
CREATE TABLE country_languages (
    country_id INTEGER NOT NULL,
    language TEXT NOT NULL,
    PRIMARY KEY (country_id, language)
);
```

Warum wichtig?

Nicht-atomare Attribute:

- erschweren Abfragen
- verhindern einfache Aggregationen
- erfordern Parsing
- erschweren Datenkonsistenz

7. Aufgabe: 2 NF

Verletzungen

Tabelle `cities`

`country_id -> country_name`

bei Schlüssel:

`(name, country_id)`

Tabelle `ports`

`country_id -> country_name`

bei Schlüssel:

`(name, country_id)`

Korrektur

`country_name` entfernen.

```
BEGIN;
    ALTER TABLE cities DROP COLUMN country_name;
    ALTER TABLE ports DROP COLUMN country_name;
COMMIT;
```

Ländernamen ausschließlich in `countries` speichern. Die Information kann über `JOIN` Statements selektiert werden:

```
SELECT * FROM cities AS c
    LEFT JOIN countries AS n ON c.country_id = n.id
    WHERE c.name = 'Berlin';
```

Konsistenz

Referenzielle Integrität über:

`FOREIGN KEY`

sicherstellen.

8. Aufgabe: 3 NF

Verletzungen

Tabelle `countries`

```
id -> capital_name
capital_name -> capital_current_governor
```

Tabelle `countries_to_rivers`

```
country_id -> country_capital_name
country_capital_name -> country_capital_area
```

Probleme

- Transitive Abhängigkeiten
- Inkonsistenzen
- Mehrfachspeicherung semantisch gleicher Informationen

Korrektur

Hauptstadtinformationen auslagern z.B. in `countries` über einen Foreign Key, der die Hauptstadt eindeutig referenziert.

9. Aufgabe: BCNF

Tabelle `city_postcodes`

Relation:

`city_postcodes(city_name, postcode, district)`

Determinant

`postcode`

Funktionale Abhängigkeit

`postcode -> district`

Kandidatenschlüssel

`(city_name, postcode)`

Problem

`postcode` ist kein Superschlüssel.

Trotzdem bestimmt `postcode` das Attribut `district`.

Damit gilt:

`X -> A`

aber `X` ist kein Superschlüssel.

Damit ist die BCNF verletzt.

Zerlegung in BCNF

```sql
CREATE TABLE postcodes (
    postcode TEXT PRIMARY KEY,
    district TEXT NOT NULL
);

CREATE TABLE city_postcodes (
    city_name TEXT NOT NULL,
    postcode TEXT NOT NULL,
    PRIMARY KEY (city_name, postcode),
    FOREIGN KEY (postcode) REFERENCES postcodes(postcode)
);
```

Begründung

In `postcodes` gilt:

`postcode -> district`

und `postcode` ist Schlüssel.

In der neuen Tabelle `city_postcodes` bleibt nur die Beziehung zwischen Stadt und Postleitzahl erhalten.

Damit bestimmt kein Nicht-Schlüssel mehr ein anderes Attribut.

Tabelle `ports`

Determinant:

`river_id`

Funktionale Abhängigkeit:

`river_id -> river_mouth_coordinates`

`river_id` ist kein Superschlüssel.

Korrektur

Flussbezogene Informationen ausschließlich in `rivers` speichern.

10. Aufgabe: Verbesserung des ursprünglichen Schemas

Problem

Der Primärschlüssel von `cities` lautet:

```
PRIMARY KEY(name, country_id)
```

Schwierigkeit

Städtenamen sind nicht eindeutig.

Beispiel:

`Hagen`

Es existieren mehrere Orte mit diesem Namen.

Konsequenzen

- natürliche Schlüssel sind instabil
- Umbenennungen problematisch
- reale Daten schwer modellierbar

Bessere Lösung

```
CREATE TABLE cities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    country_id INTEGER NOT NULL
);
```

Natürliche Schlüssel können zusätzlich über:

```
UNIQUE(name, country_id)
```

modelliert werden.